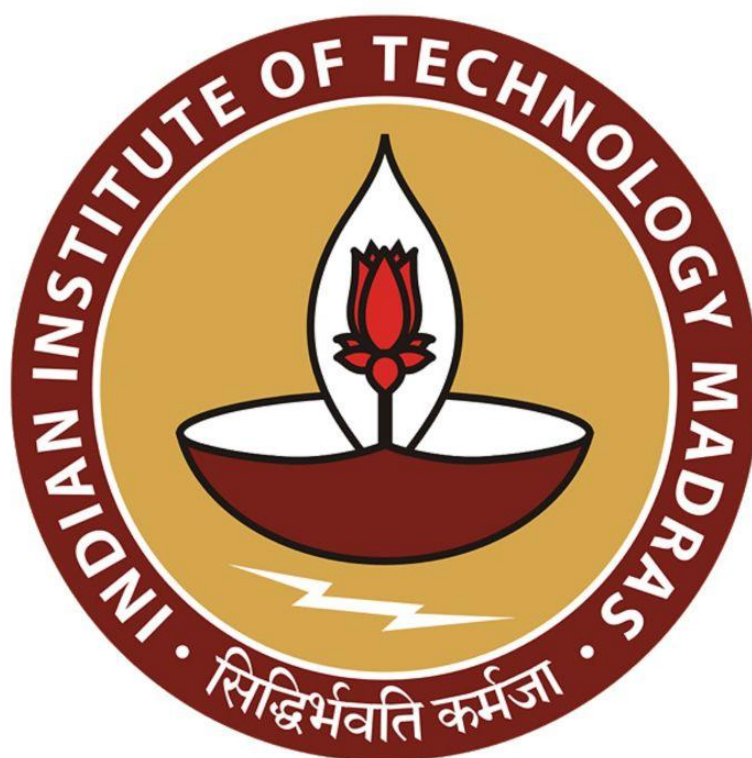
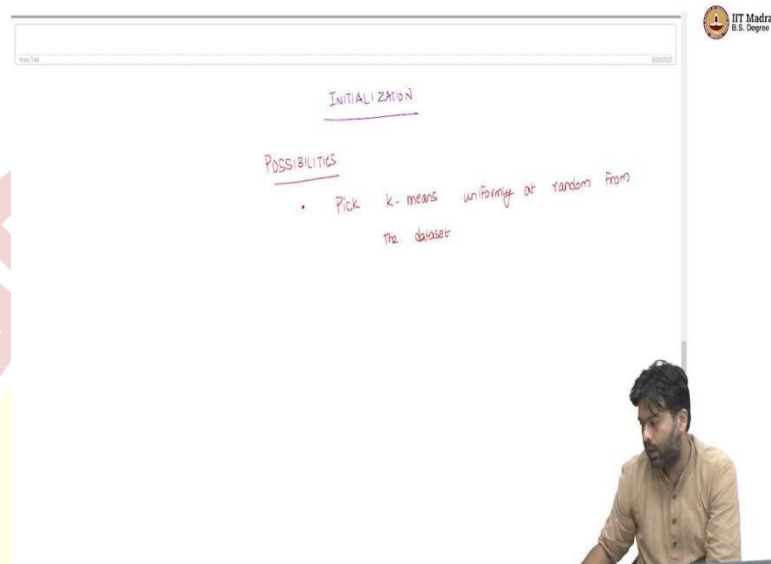




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science & Engineering
Indian Institute of Technology Madras
Initialization of Centroids, K-Means++

(Refer Slide Time: 0:14)



So, how do you initialize the Lloyd's algorithm? Well, we know that the algorithm definitely converges no matter how you initialize it, we argued that. So, our argument was agnostic to the initialization of the algorithm itself. So, we could initialize it however you want, but then some initializations might lead you to better cluster. So the question is, can we initialize it in a good way, so that we might hope to end up in good clusters. So what are some possibilities?

Well, one thing is we can simply throw data points into boxes uniformly at random, I pick a data point, I have k boxes, I will put it in a box at random. But that is not a very mindful initialization, so to say. So, we are just throwing points in boxes, it will definitely converge, the algorithm will converge, but you are not really giving that initial push to beat these local minima's, that algorithm might get stuck in when it converges. So that is so throwing points at random in boxes without really understanding the structure may not be that good an idea.

So, what do we really want, we somehow want that at the end, we know that points have the property that clusters are the property that each point is closer to its own mean. So, one thing that you can do is you can start your initialization may be in initial partition by saying that I have 1000 points in my data set, and I need to put them in 5 different boxes, I will pick 5

points uniformly at random from my data set and imagine these 5 points as my cluster centers.

It means that I am initializing, not the sets which is the cluster indicators for each data point, instead, I am initializing the means. So, I am initializing, which are going to be the means and once I fixed these means as points that are uniformly picked from the data set, now, each point will get attracted to its own closest mean, and then we will go to that box.

So, one way to do this would be to pick k-means uniformly at random from the set of data points, from the data set. So, you have 1000 data points, maybe data point number 5, data point number 72, data point number 89, 2005, 22, whatever these numbers are, those are your initial random lot, they correspond to your means.

You put them in each of these boxes separately and then for every other point, you see which of these 5 points I am closer to, and then it gets assigned to that box. So that generates initialization, where at least the points have the property that they have been chosen to be closer to a particular mean, and then they go into it. Now, that does not mean that the algorithm has converged.

So, because every point is only going to a box based on the mean, which is another data point, the single data point, lot of data points might come in, and then the means will get recomputed and then the algorithm will start from there on. And so it is but then this is a very, it is not such a bad idea to do this. And then in practice, people do this a lot of times.

And in practice, this is something that is what is typically done, if you do this, and then you will run the Lloyd's algorithm, it will converge to some partition. Now, it could solve have happened that the initial k-means that you picked as points from the dataset, you might have gotten unlucky. So, what you might want to do is, you run this with different means again.

So, you again redone on the algorithm where the initial random as is again generated, it pick 5 different means k different means and then redone the algorithm. And then you do this multiple times and you end up with the pick the clustering, which gives you the best objective value. So that is what people typically a lot of people typically do in practice. So, this is one way you could do initialization.

(Refer Slide Time: 5:04)

Pick k -means uniformly at random from the dataset

K-MEANS++

→ Choose first mean u_1 uniformly at random from $\{x_1, \dots, x_n\}$

→ for $k=2, \dots, K$

Choose u_k probability proportional to

to square

$S(x) = \min_{u_1, \dots, u_k} \sum_{i=1}^n \|x_i - u_j\|^2$

Another slightly more principled way is what is called as the k-means++ algorithm, which is just k-means algorithm with a better initialization. Here, the idea is the following, so, we still want to pick k points from the data set, which we want to think of as the k-means to begin our data algorithm with. But if you are picking them uniformly at random, you might not you are not putting effort into picking these means carefully.

So, what do we want these means to look like at the end? So, let us say we run the algorithm, algorithm converges, if you are happy, then it means that these means are as away from each other as possible, because the means are some sense representing each cluster, we are

thinking of the entire data set has been compressed into k different representatives, which are these means. And these means how to be as different as possible.

So, they are be as away from each other as possible. If they are close to each other, then it means that when the representatives are similar, well, then why should I want 2 different clusters, I could have had single cluster? So, this idea is formalized, while you initialize. You initialize carefully such that your means are in some sense as apart from each other as possible.

But then you do it with a slightly probabilistic twist and that is what will lead to this k-means++ algorithm, which I will describe now. The algorithm does the following, so it tries to not choose k-means as k different data points from your data set uniformly at random in one shot, it does not do that, whereas it does this in an iterative fashion.

What it does is the following, so first chooses the first mean. Let us call this μ_1^0 , where this naught just means that, this is the iteration number, this just means it is at initialization, μ_1^0 , uniformly at random from your set of data points, $x_1, \dots, x_n$, you have endpoints, pick one point at random and quality your first mean. So, the first mean is fixed. Now you need to choose the second mean.

So now here, this is done in an iterative fashion. So, for l equals to 2 to k , where l represents the l -th mean that you are going to pick, you are picking one at a time. So what you do is the following, so you want to pick, you want to choose μ_l^0 , that is the l -th mean at initialization. Now probabilistically, proportional to score the following score. So, you are going to give, so basically, you have a bunch of remaining data points.

So, you have already let us say, pick 5, means you want to choose the 6th mean now, you have a lot of data points, you have 1000 data points, 5 points have been already assigned as means you are in the 6th round, which means you still have 995 points from which you need to pick one. Now what we are going to do is, we are going to give a score to each of these points, and the score is a positive number.

And then you are going to pick a point with probability proportional to the score higher the score more the probability of picking that. How do you do that? You kind of can normalize this score over all the remaining points and then that will give you a probability distribution

over these remaining 995 points and then you can sample one point according to this probability distribution.

For example, instead of 995, we just had 3 points, let us say remaining points from which you need to pick one. The score of first one is 10, the score of second one is 20 and the score of third one is 30, let us say, we will talk how you get the score in a minute., let us say you had the scores 10, 20 and 30.

Now, what does it mean to say, I am probabilistically picking one point according to the scores, it means that I am going to normalize these scores to say instead of 10, 20 30, I am going to think of it as 10 divided by the sum of these scores, which is $10 + 20 + 30$, which is 60. So this will be 10 by 60, 20 by 60, 30 by 60, which is one sixth, one thirds and half and that would be the probability with which I will pick each of these data points.

So, now what are the scores? Well, now what do I want to do? I want to be as away from the means that I have been that I have chosen so far. So, let us say I have already chosen 5 means I want to pick the 6th mean, so now every point is a candidate for the 6th mean. Now, how am I going to judge this point as being good or bad with respect to the 5 means that I have picked already?

Well, I will compute the score of each of these points to the each of these means that I have already seen and see what is the smallest of these scores, which means which of these means I am closest to. The one that I am closest to, if it is, the distance to the closest mean, if it is large, then it means that I am away from all these means. So, if the closest guy is far away, then everybody else is far away.

So, I am going to give a score to each of the data points proposed as exactly the minimum distance to the means that I have seen so far. So let me put this formally, so this is like see, the score for point x is just the minimum, over all j from 1 to $l - 1$, because I already have $l - 1$, means is the l -th round. What is the distance I am thinking of $(x - \mu_j^0)^2$.

So, I am computing the distance square to each of the data points each of the which I have chosen has mean so far, and then I am seeing which is the smallest distance, which is which point I make closest to. And that distance is what I am thinking of a score for a data point. Now, every data point gets a score, so for all x , this is for all x in the data set, I can compute the score.

And now I will now sample the next mean, the next as a data point according to the score, so in a probabilistic fashion, which I explained earlier. So, now you are going to normalize these scores over all the data points and then you are going to probabilistically sample one data point like this. So, if x_1, x_2, x_3 are the remaining data points, and x_1 score is 10, x_2 score is 20, x_3 is 30, then x_1 , the probability that x_1 gets chosen as 1 by 6 , which is 10 by 60 and x_2 gets chosen as 1 by 3 , which is 20 by 60 and x_3 gets chosen as 1 by 2 , which is 30 by 60 .

So, I want to sample according to x . So, this is the k-means++ way of doing initialization. One might wonder why am I doing this in a probabilistic way, why cannot I simply do this in a deterministic way? I want the means to be as away from things as possible. Why cannot they simply pick as of the next mean as the 1, whose minimum distance to the ones that have chosen so far is maximum?

I could have done that, so which means in this case, if there are 3 points, 10, 20 and 30, well I would pick x_3 simply because x_3 minimum distance to whatever that has been chosen, is much higher than the 2 which means it is much far apart than the means that I have seen so far. Well, what we have seen is that x_3 still gets the highest probability of being chosen in k-means++, but it may not necessarily be chosen, it could be x_2 also.

And you need this probabilistic way of doing this, instead of deterministic way, because you can show some kind of guarantee for this algorithm.

(Refer Slide Time: 13:45)

→ for $k = 2, \dots, K$

Choose μ_k probabilistically proportional to score

for score

$S(x) = \min_{j=1, \dots, k} \|x - \mu_j\|^2$

x_1, x_2, x_3	Score	Normalized
10	10	$\frac{1}{6}$
20	20	$\frac{2}{3}$
30	30	$\frac{1}{2}$

Guarantee

$$\mathbb{E} \left[\sum_{i=1}^n \|x_i - \mu_k\|^2 \right] \leq O(1/k) \left[\min_{\mu_1, \dots, \mu_k} \sum_{i=1}^n \|x_i - \mu_k\|^2 \right]$$

min. sum of squares of

IIT Madras
B.S. Degree

INITIALIZATION

POSSIBILITIES

- Pick k-means uniformly at random from the dataset

K-MEANS++

→ Choose first mean μ_1 uniformly at random from $\{x_1, \dots, x_n\}$

So, we I will brief as upon this guarantee, what this guarantee, what can you guarantee for this algorithm is as follows. And this to argue this guarantee, you cannot put down a deterministic algorithm, because once you put down a deterministic algorithm, you can come up with an adversarial data set where this guarantee might fake.

So, you want in some sense, an average guarantee and so you make the algorithm randomized. But then you do the most natural thing where you kind of introduced probability where the deterministic choice will get the highest probability, but then still other people might also get some probability so that on an average, you can make some kind of guarantees.

So, I will briefly touch upon the guarantee. So, the guarantee looks something like this. We will not go into the details or argue why this is true in this course, but then it is good to know that there is this underlying guarantee available. So, the expected value of $\sum_{i=1}^n (x_i - \mu_{z_i})^2$, which is the, this is just the objective of the partition that the algorithm results after a run k-means.

I do this initialization run Lloyd's, and then I end up with a partition and I can compute its objective value. Now, that objective values are random quantity, because my initialization was random. The Lloyd's algorithm itself is not random, so once you fix an initialization, everything else is deterministic in the Lloyd's algorithm.

But because the initialization is random, the answer final answer is also going to be random. So, which means I can ask over the randomness of the over randomness of algorithm, which is the initialization, how does this on an average how am I doing with respect to the objective function. What I really want or care about is the best possible thing, So, minimum $\sum_{i=1}^n (x_i - \mu_{z_i})^2$.

So, this is the best possible that I want, I want that partition, which gives me the least possible objective value, I may not be able to get it and that is an NP hard problem to get that. But what I am saying is here, k-means++ guarantees is that you may not be able to achieve that partition, which gets this but then on an average, you are not going to be too far away from that partition.

In some sense, you are going to be something like some constant times this quantity, something like this. Basically, what it is saying is that, let us say the best possible objective function gives me some value of 10. On an average, I am saying that the partition that k-means++ will result in is not going to be more than let us say, the objective value is not going to be more than let us say 5 times 10.

So, it is got to be more than it, but then it will not be too far away also, on an average, that is the kind of guarantee that you can give. The specifics of the guarantee, or how this comes about is beyond the scope of the course, we will not prove that. But it is good to know that, by doing reasonable initialization, you might end up in partitions, which are not too bad.

The downside of doing this is though, that you need to spend a lot of time doing this initialization, So, because if you had a billion points, and you want to choose 10 different means, now every time you need to compute these scores, and then you need to, do a probability, convert that to a probability sample, and so on, you can do some tests to make that faster, but then still, you need to go through this thing of doing this k different times.

So, computationally, this might be less efficient, or might take more time than just, our previous method, which is just pick k-means uniformly at random, and then try it out and try it multiple times and see what works. So that is faster, the uniformly at random method is faster, but then it is not, I would say principle. So it is not pushing your means in certain direction. But typically, in practice, especially when you have high dimensional data, people also use the uniform one extensively.

So now, it depends on how much time your algorithm, I mean, you have to run your algorithm, how much resources you have and all that will make you this, I mean, you can make a decision based on that. Nevertheless, k-means++ is a solid way to initialize your Lloyd's algorithm. So that is, what I wanted to comment about the initialization part. So, the final thing that we will talk about is the choice of k which we will see next.

